Final Exam Review

Zizhuo Wang

School of Data Science (SDS)
The Chinese University of Hong Kong, Shenzhen

May 6, 2023

Final Exam Logistics

- Time: May 17, Wednesday, 8:30am-10:30am. Location: TBA.
- Two pieces of notes are allowed (double sided, normal size)
- No electronic device is allowed
- Sample final exam and its solution are posted. The format of the final exam will be similar. You are strongly encouraged to work on it to prepare for the final exam.
- Homework 9 due on May 10. Solution will be posted afterwards.
- Office hour: May 9, 12:00pm-1:30pm or by appointment.

High-Level Review

The final exam will mainly cover:

- Nonlinear optimization
- Integer optimization

There will not be any question that is solely related to linear optimization. However, some questions may require knowledge about LP (and general knowledge about optimization that we learned in the first half of the semester)

Nonlinear Optimization

We mainly learned the three topics for nonlinear optimization

- Optimality (KKT) conditions
- Convexity of problems
- Algorithms

Optimality Conditions

For unconstrained problems

- First-order necessary condition
- Second-order necessary condition
- Second-order sufficient condition

For constrained problems

- We mainly consider the first-order necessary condition - KKT conditions

What you are expected to know about those conditions

- Be able to derive those conditions
- Fully understand the meaning of those conditions
- Can use those conditions to derive useful insights or solve the original problem (if possible)

Unconstrained Problems

Theorem (First-Order Necessary Condition)

If $\mathbf{x}^*$ is a local minimizer of $f(\cdot)$ for the unconstrained problem, then we must have $\nabla f(\mathbf{x}^*) = 0$.

Theorem (Second-Order Necessary Condition)

If $\mathbf{x}^*$ is a local minimizer of $f(\cdot)$ for an unconstrained problem, then we must have

1. $\nabla f(\mathbf{x}^*) = 0$;
2. $\nabla^2 f(\mathbf{x}^*)$ *is positive semi-definite.*

Theorem (Second-Order Sufficient Conditions)

Let f be second-order continuously differentiable. If $\mathbf{x}^*$ satisfies:

1. $\nabla f(\mathbf{x}^*) = 0$
2. $\nabla^2 f(\mathbf{x}^*)$ *is positive definite*

Then $\mathbf{x}^*$ is a local minimum of f for the unconstrained problem.

Check Positive Semi-Definiteness

For one dimensional problem, positive semi-definiteness/definitess is equivalent to nonnegativeness/strictly positiveness.

For higher dimensional problem, it is equivalent to all the eigenvalues are nonnegative/positive.

In practice, one typically checks the determinant of all the principal submatrices.

- Can check for 2×2 matrices
- Similar for negative semi-definite/definite matrices

Our conditions are derived for minimization problem.

- For maximization problem, it is just the opposite direction

Some Additional Terminologies

Definition (Stationary Points)

We call a point $\mathbf{x}$ satisfying $\nabla f(\mathbf{x}) = 0$ a *stationary point* of $f(\cdot)$.

- FONC characterizes all stationary points.

Definition (Saddle Points)

Suppose that $\mathbf{x}$ is a stationary point ($\nabla f(\mathbf{x}) = 0$), but it is neither a local maximizer or a local minimizer, then we call $\mathbf{x}$ a *saddle point*.

- If $\mathbf{x}$ is a stationary point, and the Hessian matrix $\nabla^2 f(\mathbf{x})$ is neither positive semi-definite nor negative semi-definite (we call such matrices indefinite matrices), then $\mathbf{x}$ must be a saddle point.

KKT Conditions

KKT conditions is the first-order necessary condition for general optimization problem:

$$
\begin{aligned}
\text{minimize}_{\mathbf{x}} \quad & f(\mathbf{x}) \\
\text{s.t.} \quad & g_i(\mathbf{x}) \geq 0 \quad i = 1, ..., m \\
& h_i(\mathbf{x}) = 0 \quad i = 1, ..., p \\
& \ell_i(\mathbf{x}) \leq 0 \quad i = 1, ..., r \\
& x_i \geq 0 \quad i \in M \\
& x_i \leq 0 \quad i \in N \\
& x_i \text{ free} \quad i \notin M \cup N
\end{aligned}
$$

Construct KKT Conditions

The first step is to construct the *Lagrangian* of this problem defined as follows:

1. We associate each constraint (not including the sign constraints of $\mathbf{x}$) with a *Lagrangian multiplier*

$$\begin{aligned} g_i(\mathbf{x}) \geq 0 \quad &\cdots \quad \lambda_i \\ h_i(\mathbf{x}) = 0 \quad &\cdots \quad \nu_i \\ \ell_i(\mathbf{x}) \leq 0 \quad &\cdots \quad \eta_i \end{aligned}$$

2. We define the *Lagrangian* of this problem by

$$L(\mathbf{x}, \lambda, \nu, \eta) = f(\mathbf{x}) + \sum_{i=1}^{m} \lambda_i g_i(\mathbf{x}) + \sum_{i=1}^{p} \nu_i h_i(\mathbf{x}) + \sum_{i=1}^{r} \eta_i \ell_i(\mathbf{x})$$

Lagrangian Multipliers

We require that the multipliers associated with ≥ 0 constraints (λ_i's) be negative, the multipliers associated with equality constraints (ν_i's) be free and the multipliers associated with ≤ 0 constraints (η_i's) be positive.

- $\lambda_i \leq 0$, ν_i free and $\eta_i \geq 0$ form one part of the KKT conditions.
- We call them the "dual feasibility" conditions

KKT from Lagrangian

Now we derive the KKT conditions from the Lagrangian. We first take derivative to all x_i. We set the derivative to be

1. ≥ 0 if $x_i \geq 0$
2. ≤ 0 if $x_i \leq 0$
3. $= 0$ if x_i is free

This forms one part of the KKT conditions. We call them the main conditions.

KKT from Lagrangian

Finally we have a set of complementarity conditions:

- Each multiplier is complementary to the constraint it is associated to, i.e.,

$$\begin{aligned} \lambda_i \cdot g_i(\mathbf{x}) &= 0, \quad \forall i \\ \nu_i \cdot h_i(\mathbf{x}) &= 0, \quad \forall i \\ \eta_i \cdot \ell_i(\mathbf{x}) &= 0, \quad \forall i \end{aligned}$$

  The second condition is usually omitted since the feasibility of $\mathbf{x}$ already guarantees it.

- Each x_i is complementary to $\nabla_{x_i} L(\mathbf{x}, \lambda, \nu, \eta)$, i.e.,

$$x_i \cdot \nabla_{x_i} L(\mathbf{x}, \lambda, \nu, \eta) = 0$$

  This condition is usually omitted if x_i is a free variable since in that case the main condition already guarantees it.

KKT Conditions

1. Main Condition

$$\nabla_j f(\mathbf{x}) + \sum_{i=1}^{m} \lambda_i \nabla_j g_i(\mathbf{x}) + \sum_{i=1}^{p} \nu_i \nabla_j h_i(\mathbf{x}) + \sum_{i=1}^{r} \eta_i \nabla_j \ell_i(\mathbf{x}) \begin{cases} \geq \\ \leq \\ = \end{cases} 0, \quad \begin{cases} \text{if } x_j \geq 0 \\ \text{if } x_j \leq 0 \\ \text{if } x_j \text{ free} \end{cases}$$

2. Dual Feasibility

$$\lambda_i \leq 0, i = 1, ..., m; \quad \eta_i \geq 0, i = 1, ..., r$$

3. Complementarity

$$\lambda_i \cdot g_i(\mathbf{x}) = 0, \forall i; \quad \eta_i \cdot \ell_i(\mathbf{x}) = 0, \forall i; \quad x_i \cdot \nabla_{x_i} L(\mathbf{x}, \lambda, \nu, \eta) = 0, \forall i$$

4. Primal Feasibility

$$g_i(\mathbf{x}) \geq 0, h_i(\mathbf{x}) = 0, \ell_i(\mathbf{x}) \leq 0, \forall i$$
$$x_i \geq 0, \forall i \in M; \quad x_i \leq 0, \forall i \in N$$

KKT Conditions

For maximization problem, we can first convert it into a minimization problem and then derive the KKT conditions

How did we get those KKT conditions?

- We considered the feasible direction set and the descent direction set at $\mathbf{x}$
- In order for $\mathbf{x}$ to be optimal, we need that the intersection between the feasible direction set and the descent direction set at $\mathbf{x}$ is an empty set

Can use KKT conditions to solve optimization problems when possible

Convexity

You need to understand the following concepts: convex set, convex/concave function, convex constraint and convex optimization problem

Definition (Convex Set)

S is a convex set if and only if for any $\mathbf{x}, \mathbf{y} \in S$ and any $0 \leq \alpha \leq 1$, $\alpha\mathbf{x} + (1 - \alpha)\mathbf{y} \in S$.

To check whether a set is a convex set or not, most of the time, just check the definition.

Convex Functions

Definition (Convex function)

A function f on Ω is said to be *convex* if for any $\mathbf{x}_1, \mathbf{x}_2 \in \Omega$ and any $0 \leq \alpha \leq 1$,

$$f(\alpha \mathbf{x}_1 + (1-\alpha)\mathbf{x}_2) \leq \alpha f(\mathbf{x}_1) + (1-\alpha) f(\mathbf{x}_2)$$

Definition (Concave function)

We call f a concave function if and only if $-f$ is a convex function or for any $\mathbf{x}_1, \mathbf{x}_2$ and $0 \leq \alpha \leq 1$,

$$f(\alpha \mathbf{x}_1 + (1-\alpha)\mathbf{x}_2) \geq \alpha f(\mathbf{x}_1) + (1-\alpha) f(\mathbf{x}_2)$$

- You should be able to check whether a function is convex or concave or neither

Check a Function is Convex

Proposition

If $f(\mathbf{x})$ is second order differentiable. Then f is convex if and only if its Hessian matrix is positive semi-definite (PSD) throughout the defined region

- In one dimensional case, this means the second-order derivative is non-negative
- Taking second order derivative is usually the easiest way to test convexity

Proposition

If $f(\mathbf{x})$ is second order differentiable. Then f is concave if and only if its Hessian matrix is negative semi-definite throughout the defined region

- In one dimensional case, this means the second-order derivative is non-positive

Check a Function is Convex

Proposition

If $a_1, ..., a_m \geq 0$, and $f_1, ..., f_m$ are convex (concave) functions, then $a_1 f_1 + \cdots + a_m f_m$ is a convex (concave) function.

Proposition

If $f : \mathbb{R}^n \to \mathbb{R}$ is convex (concave), then $g(\mathbf{x}) = f(A\mathbf{x} + \mathbf{b})$ is convex (concave).

Proposition

If $f_1, ..., f_m$ are convex functions, then $f(\mathbf{x}) = \max\{f_1(\mathbf{x}), ..., f_m(\mathbf{x})\}$ is a convex function (this can be extended to uncountably many).

Proposition

If $f_1, ..., f_m$ are concave function, then $f(\mathbf{x}) = \min\{f_1(\mathbf{x}), ..., f_m(\mathbf{x})\}$ is a concave function (this can be extended to uncountably many).

- If none of the above method applies, one could also try to check by definition.

Convex Constraint

Definition (Convex Constraint)

We call a constraint a convex constraint if the feasible region defined by the constraint is a convex set

In practice, we frequently use the following (sufficient) criterions

- If a constraint is $g(\mathbf{x}) \leq 0$, and $g(\mathbf{x})$ is a convex function
- If a constraint is $g(\mathbf{x}) \geq 0$, and $g(\mathbf{x})$ is a concave function
- Any linear constraint

Otherwise, one may need to check the definition

Convex Optimization Problems

A convex optimization problem is an optimization problem that:

- Either minimizing a convex function or maximizing a concave function
- The constraints are convex constraints

Implications of convexity

- For a convex optimization problem, any local optimum is also global optimum.
- KKT condition is sufficient for global optimality

In optimization world, whether or not a problem is a convex optimization problem is the main watershed in determining whether a problem is easy or hard

Convex Optimization Problems

Sometimes, you need to do some transformations to make a problem convex

Common ways to do the transformation:

- Monotone transformation of objective value and/or the constraint
- Substitute variables to remove nonlinearity

You should be able to identify whether an optimization problem is a convex one, and if not, be able to convert it into a convex one if possible.

Algorithms for Nonlinear Optimization

In practice, we need to have a detailed procedure that can guarantee to solve the problem efficiently - Algorithms

For one dimensional unconstrained problems

- Bisection method
- Golden section method

For high dimensional unconstrained problems

- Gradient descent method
- Newton's method

For constrained problems

- Gradient projection method

You should understand how each of these algorithms works, their procedures and performance

One Dimensional Problem

Bisection method

- Try to find the root of $f'(x)$
- Need to know the derivative

Golden section method

- Directly find the optimal solution
- Don't need derivative information

Gradient Descent and Newton's Method

Both methods are iterative method and have the same framework

Start from a point $\mathbf{x}^0$, set a tolerance level $\epsilon > 0$ and $k = 0$.

1. If $||\nabla f(\mathbf{x}^k)|| < \epsilon$, then output $\mathbf{x}^k$, otherwise continue;
2. Compute the search direction $\mathbf{d}^k$.
3. Choose step size α_k by backtracking line search
4. Set $\mathbf{x}^{k+1} = \mathbf{x}^k + \alpha_k \mathbf{d}^k$, $k = k + 1$, go back to step 1

In gradient descent method

$$\mathbf{d}^k = -\nabla f(\mathbf{x}^k)$$

In Newton's method

$$\mathbf{d}^k = -(\nabla^2 f(\mathbf{x}^k))^{-1} \nabla f(\mathbf{x}^k)$$

Convergence of the Gradient Method

Theorem

Under some technical conditions on $f(\mathbf{x})$, the gradient method has linear convergence:

$$f(\mathbf{x}^k) - p^* \leq c^k (f(\mathbf{x}^0) - p^*)$$

where p^* is the (local) optimal value, $c < 1$ is some constant

Therefore, gradient method is guaranteed to converge to a local minimum (regardless of the starting point)

Corollary

If $f(\mathbf{x})$ is convex, then gradient method is guaranteed to converge to the global minimum.

Newton's Method

High-level idea:

- Approximate the objective function by a quadratic function, and use the minimizer of the quadratic function as the next iterate.

When the initial point is close enough to the optimal point, then Newton's method has quadratic convergence

$$||\mathbf{x}^{k+1} - \mathbf{x}^*|| \leq C||\mathbf{x}^k - \mathbf{x}^*||^2$$

If $f(\mathbf{x})$ is strictly convex, then by using backtracking line search, Newton's method also converges to a local optimal solution.

Tradeoff between Newton and Gradient Methods

- Newton's method converges much faster (quadratic convergence), however, it requires the information of the second-order derivative/Hessian matrix, and within each iteration, it is much more computationally expensive (needs to compute the Hessian and matrix inverse)
- Gradient method is much faster in each iteration and needs less space, however, it takes much more iterations to converge

There are many numerical improvements that one can make but the main tradeoff remains

Gradient Projection Method

High-level idea:

- We try to use the same idea as the gradient descent method
- However, the solution may not be feasible
- To solve this problem, each time, we project the negative gradient direction to the feasible direction set
- We only consider the problem with constraint $A\mathbf{x} = \mathbf{b}$.

Theorem

The projection of $\mathbf{x}$ *to the region* $A\mathbf{x} = 0$ *is given by* $P_A\mathbf{x}$*, where* $P_A = I - A^T(AA^T)^{-1}A$

Procedure of Gradient Projection Method

Start with any point $\mathbf{x}^0$. Set $k = 0$ and stopping criterion $\epsilon > 0$.
Define $P_A = I - A^T(AA^T)^{-1}A$

1. Check $||P_A \nabla f(\mathbf{x}^k)||$. If $||P_A \nabla f(\mathbf{x}^k)|| \leq \epsilon$, stop and output $\mathbf{x}^k$. Otherwise, continue to Step 2
2. Let $\mathbf{d}^k = -P_A \nabla f(\mathbf{x}^k)$ as the search direction
3. Use backtracking line search to find the stepsize α_k
4. Let $\mathbf{x}^{k+1} = \mathbf{x}^k + \alpha_k \mathbf{d}^k$, let $k = k + 1$. Go back to step 1

- The only difference with the gradient or Newton's method is the search direction

Gradient projection method has the same convergence result as the gradient method: It converges to a local minimum in a linear speed and converges to the global optimum if the problem is convex.

Integer Optimization

- Formulation
- The role of LP relaxation
- Branch-and-bound algorithm

Formulate IP Problems

It is important to be able to formulate a problem using integer programming formulation

- Familiar with all example types in the lecture, homework, etc

Being able to use integer variables to represent relations between variables

- The on-off type of relation - only if x is 1, y can be nonzero, as in the facility location problem
- The "or" type of relation - one of two cases holds, use a binary variable and a big M for each case to handle it
- This can be extended to a certain number of cases hold among many cases

Hardness of IP

In general, IP is very hard to solve

- No polynomial-time algorithm is known to solve general IP

Linear programming (LP) relaxation

- Relax (ignore) all integer constraints and solve the linear program
- Relaxation of binary constraint $x \in \{0, 1\}$: $0 \leq x \leq 1$
- Provide bounds on the optimal value of IP (upper bound for maximization and lower bound for minimization)
- It can also be used to provide performance bound for any *feasible* solution to the IP
- Integrality gap

$$|v^{LP} - v^{IP}|$$

LP Relaxation

Usually solving the LP relaxation does not give one the optimal solution to IP

- Simply round a fractional solution to an integral solution does not work
- However, if the optimal solution to the LP relaxation is integral, then it must be the optimal solution to IP as well
- Sufficient conditions to guarantee that an LP always has an integral optimal solution – Totally Unimodularity (TU)
- Being able to check TU using the condition introduced in class

If constraint A is TU and $\mathbf{b}$ is an integer vector, then

- All basic feasible solutions are integral
- Solving the LP will solve the IP

One Sufficient Condition for TU

Theorem

Let A be an $m \times n$ matrix. Then the following conditions together are sufficient for A to be totally unimodular:

1. *Every column of A contains at most two non-zero entries;*
2. *Every entry in A is 0, $+1$, or -1;*
3. *The rows of A can be partitioned into two disjoint sets B and C such that*
   (a) *If two non-zero entries in a column of A have the same sign, then the row of one entry is in B, and the row of the other in C;*
   (b) *If two non-zero entries in a column of A have opposite signs, then the rows are both in B, or both in C.*

Branch-and-Bound Method

High-level idea:

- Branching: Divide the feasible region into smaller ones, solve each of them and combine them to find the optimal solution
- Bounding: Use bounds (LP optimal value and feasible solutions obtained) to reduce the number of branches we need to consider

Branch-and-Bound Method

Branching Procedures:

1. Solve the LP relaxation
   - If the optimal solution is integral, then it is optimal to IP
   - Otherwise go to step 2
2. If the optimal solution to the LP relaxation is $\mathbf{x}^*$ and x_i^* is fractional, then branch the problem into the following two:
   - 2.1 One with an added constraint that $x_i \leq \lfloor x_i^* \rfloor$
   - 2.2 One with an added constraint that $x_i \geq \lceil x_i^* \rceil$
3. For each of the two problems, use the same method to solve them, and get optimal solution $\mathbf{x}_1^*$ and $\mathbf{x}_2^*$ with optimal value v_1^* and v_2^*
   - Compare and get the optimal solution

Branch-and-Bound Method

Bounding procedures (for maximization):

- Any LP relaxation solution can provide an upper bound for each node in the branching process
- Any feasible solution to the IP can provide a lower bound for the entire problem
- We call the best integer solution obtained for the problem the "incumbent solution"

When at a certain node, the optimal value of the LP relaxation of this branch is even less than the current lower bound (the objective value of the incumbent solution). Then we should abandon this branch (also called prune or fathom that branch)

- No better solution can be obtained from exploring this branch

Bounding is very important for branch-and-bound, it is the key to make it efficient (and practical)

Integer Program Summary

- Be able to formulate simple IP problems
- Can discuss the relationship between the LP relaxation and the IP problem (e.g., what information does the LP relaxation provide about the IP)
- Be able to use branch-and-bound method to solve small IP problems